IPA Project Report: Sequential RV64I RISC-V Processor

Sukeerthi (2024102074)
Chandralekhya (2024102034)
Padmanjali (2024102047)

Abstract

This report presents a Verilog-based sequential RV64I RISC-V processor designed using a modular single-cycle architecture. The processor supports key instructions such as `add`, `sub`, `addi`, `and`, `or`, `ld`, `sd`, and `beq`. The system integrates modules such as the program counter, instruction memory, register file, control unit, ALU, and data memory. Functionality is verified through simulation using RISC-V assembly programs.

I. Introduction

This project implements a **single-cycle 64-bit RISC-V processor** using Verilog. The sequential architecture executes each instruction—fetch, decode, execute, memory access, and write-back—within a single clock cycle. The processor supports a required subset of the RV64I instruction set, including arithmetic, logical, load/store, and branch operations (`add`, `sub`, `addi`, `and`, `or`, `ld`, `sd`, `beq`).

The design follows a **modular structure**, with each block (Program Counter, Instruction Memory, Register File, Control Unit, Immediate Generator, ALU Control, ALU, and Data Memory) developed and tested independently before integration. Verification is conducted through simulation by running RISC-V test programs and analyzing the final register file state.

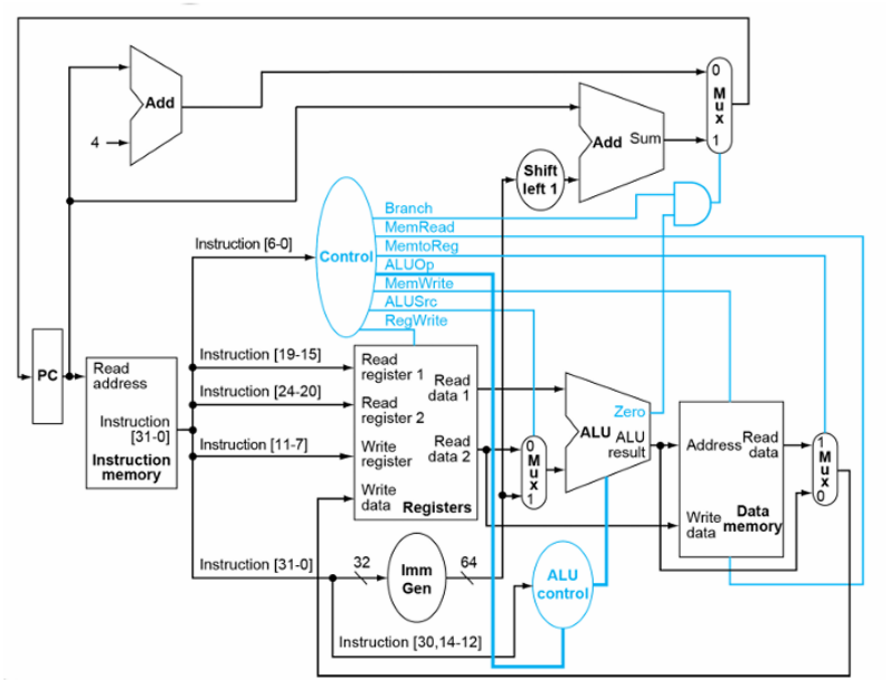


Fig. 1: Sequential datapath

II. System Overview (Main Idea)

The proposed design follows the standard single-cycle RISC-V datapath described in undergraduate computer architecture texts, adapted to a 64-bit RV64I subset. In this architecture, every instruction flows through the same sequence of operations—fetch, decode, execute, memory access, and write-back—within a single clock cycle. This provides a clear and modular view of the processor while keeping the control logic relatively simple.

The processor is organized around the following core modules:

- **Program Counter (PC):**
  The Program Counter holds the 64-bit address of the current instruction. On each rising clock edge, it is updated to either `PC + 4` for normal sequential execution or a branch target when a branch instruction is taken. The PC serves as the entry point of the fetch stage and ensures correct flow of instruction execution.
- **Instruction Memory:**
  Instruction Memory stores the program in the form of 32-bit machine instructions. The PC value is used as an address, and the corresponding instruction is returned to the decode stage. Memory contents are preloaded using external files as specified by the project requirements.
- **Register File:**
  The Register File implements 32 general purpose 64-bit registers with two read ports and one write port. Registers `rs1` and `rs2` are read during decoding, and the destination register `rd` is written during the write-back stage if `RegWrite` is enabled. Register `x0` is hardwired to zero.
- **Main Control Unit:**
  The Control Unit decodes the 7-bit opcode of the instruction and generates control signals such as `RegWrite`, `MemRead`, `MemWrite`, `MemtoReg`, `Branch`, and `ALUSrc`. These signals determine how data flows through the datapath and which functional units are activated for the instruction.
- **Immediate Generator:**
  The Immediate Generator extracts and sign-extends immediate fields from I-type, S-type, and B-type instructions based on the RISC-V format. Correct immediate generation is essential for arithmetic immediates, load/store address calculations, and branch offsets.

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

- **ALU Control and ALU:**

  The ALU Control Unit combines the high-level `ALUOp` signal from the Control Unit with the `funct3` and `funct7` fields of the instruction to determine the specific ALU operation—such as addition, subtraction, AND, or OR. The ALU performs arithmetic and logical operations required by the instruction. It also generates a `zero` flag that is used by branch instructions to determine whether a branch should be taken.
- **Data Memory:**
  Data Memory provides load and store functionality for 64-bit data. For load instructions, it returns

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Fig. 2: The effect of each of the control signals

data to the write-back stage through the `MemtoReg` multiplexer. For store instructions, it receives data from the register file and writes it to the specified memory address.

- **Supporting Multiplexers and Adders:**
  Multiplexers select the appropriate data at each stage of the datapath, choosing between register values, immediates, ALU outputs, or memory data. They prevent the need for separate hardware for every instruction and keep the design compact.
  Adders are used to compute `PC + 4` for sequential execution and to generate branch target addresses using the sign-extended immediate. Together, they manage program flow and enable correct transitions between instructions.

Conceptually, instruction processing can be divided into the following five stages:

1) **Instruction Fetch (IF):** The PC is used to fetch the next instruction from instruction memory.
2) **Instruction Decode (ID):** The instruction fields are decoded, register operands are read, and the immediate value is generated.
3) **Execute (EX):** The ALU performs the required arithmetic or logical operation, or computes an effective address or branch target.
4) **Memory Access (MEM):** For load and store instructions, data memory is accessed using the address from the ALU.
5) **Write Back (WB):** The result from either the ALU or data memory is written back into the destination register.

In the implemented single-cycle processor, all of these stages are completed within a single clock period for each instruction. This system-level view forms the basis for the detailed module descriptions and verification methodology discussed in the following sections.

III. Verification Methodology

A structured verification approach was followed to ensure the correctness of every module and the overall sequential RV64I processor. Verification was performed in three stages: module-level testing, top-level processor testing, and complete program execution.

A. Module-Level Testing

Each hardware block was independently verified using dedicated Verilog testbenches. The Program Counter, Register File, Immediate Generator, ALU Control, ALU, and Data Memory were tested with representative input patterns to confirm correct functionality. This stage ensured that each component behaved according to the RV64I specification before integrating the modules together.

B. Top-Level Testbench Setup

A unified testbench was created to simulate the complete processor. The testbench generated a clock signal, applied an initial reset pulse, and preloaded the instruction memory using the external file specified by the project guidelines. During simulation, the processor executed one instruction per clock cycle. The testbench also monitored the Program Counter, register values, ALU operations, and memory accesses to validate correct datapath operation.

C. Test Programs Used

Processor functionality was validated by executing multiple RISC-V assembly programs. These programs included arithmetic computations (`add`, `sub`, `addi`), logical operations (`and`, `or`), load and store sequences (`ld`, `sd`), and control-flow instructions (`beq`). Simple sequences such as register updates, memory accesses, and branch loops were used to verify correct execution of all supported instructions.

D. Output Verification

At the end of execution, the processor generated a `register_file.txt` dump containing the final values of all 32 registers, followed by the number of clock cycles taken. These values were compared against manually calculated expected outputs to confirm correct behavior. Intermediate waveforms were inspected using GTKWave to analyze signal transitions and identify datapath mismatches.

E. Simulation Tools

All Verilog code was compiled and simulated using **Icarus Verilog (iverilog)**. Signal traces were viewed using **GTKWave**, which helped verify control signals, ALU outputs, memory accesses, and Program Counter updates for each executed instruction.

IV. Implementation and Simulation Results (Module-Wise)

During the development of the sequential RV64I processor, each module was implemented and verified independently before integrating the complete datapath. This modular testing approach ensured that functional correctness was maintained at every stage of the processor.

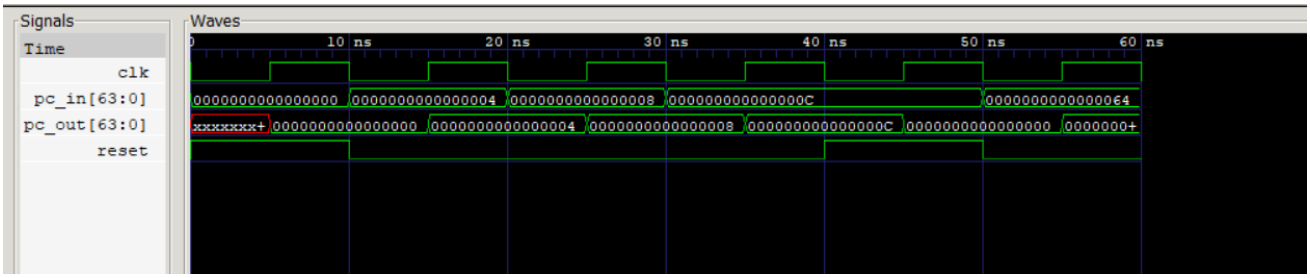


Fig. 3: PC output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog pc.v pc_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
Time=0 | reset=1 | pc_in=0 | pc_out=x
Time=5000 | reset=1 | pc_in=0 | pc_out=0
Time=10000 | reset=0 | pc_in=4 | pc_out=0
Time=15000 | reset=0 | pc_in=4 | pc_out=4
Time=20000 | reset=0 | pc_in=8 | pc_out=4
Time=25000 | reset=0 | pc_in=8 | pc_out=8
Time=30000 | reset=0 | pc_in=12 | pc_out=8
Time=35000 | reset=0 | pc_in=12 | pc_out=12
Time=40000 | reset=1 | pc_in=12 | pc_out=12
Time=45000 | reset=1 | pc_in=12 | pc_out=0
Time=50000 | reset=0 | pc_in=100 | pc_out=0
Time=55000 | reset=0 | pc_in=100 | pc_out=100
pc_tb.v:54: $finish called at 60000 (1ps)
```

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog inst_mem.v inst_mem_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
addr=0 instr=00500113
addr=4 instr=00a00193
addr=8 instr=003100b3
addr=12 instr=40310133
addr=16 instr=0031f233
addr=20 instr=0041f2b3
addr=24 instr=00416333
addr=28 instr=003163b3
addr=32 instr=0012b023
addr=36 instr=0002b503
addr=40 instr=0062bc23
addr=44 instr=0182b583
addr=48 instr=00520463
addr=52 instr=00000063
addr=56 instr=00a086b3
addr=60 instr=00000000
addr=64 instr=00000000
inst_mem_tb.v:14: $finish called at 85 (1s)
```

Fig. 4: Instruction Memory output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog reg_file.v reg_file_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
VCD info: dumpfile reg_file.vcd opened for output.
After reset: rd1=0000000000000000 rd2=0000000000000000 (expected both 0000000000000000)
Read x1,x2: rd1=000000000000000a rd2=00000000000000ff
Expected   : rd1=000000000000000A rd2=00000000000000FF
Read x3,x0: rd1=deadbeefdeadbeef rd2=0000000000000000
Expected   : rd1=DEADBEEFDEADBEEF rd2=0000000000000000
reg_file_tb.v:104: $finish called at 61000 (1ps)
```

Fig. 5: Register output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog imm_gen.v imm_gen_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
===== IMM_GEN TEST =====
I-TYPE ADDI 10      imm_out = 10 (hex=000000000000000a)  EXPECTED: 10
I-TYPE ADDI -5      imm_out = -5 (hex=fffffffffffffffb)  EXPECTED: -5
LOAD 16(x2)         imm_out = 16 (hex=0000000000000010)  EXPECTED: 16
S-TYPE SD 20(x4)    imm_out = 20 (hex=0000000000000014)  EXPECTED: 20
S-TYPE SD -8(x4)    imm_out = -4 (hex=fffffffffffffffc)  EXPECTED: -4
B-TYPE BEQ +16      imm_out = 8 (hex=0000000000000008)  EXPECTED: 8
B-TYPE BEQ -4       imm_out = -4 (hex=fffffffffffffffc)  EXPECTED: -4
===== TEST COMPLETE =====
imm_gen_tb.v:77: $finish called at 35000 (1ps)
```

Fig. 6: Immediate Generator output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog control.v control_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
VCD info: dumpfile control_wave.vcd opened for output.

=== R-TYPE (ADD) ===
opcode=0110011
Branch=0 MemRead=0 MemtoReg=0 ALUOp=10 MemWrite=0 ALUSrc=0 RegWrite=1

=== I-TYPE (ADDI) ===
opcode=0010011
Branch=0 MemRead=0 MemtoReg=0 ALUOp=00 MemWrite=0 ALUSrc=1 RegWrite=1

=== LOAD (LD) ===
opcode=0000011
Branch=0 MemRead=1 MemtoReg=1 ALUOp=00 MemWrite=0 ALUSrc=1 RegWrite=1

=== STORE (SD) ===
opcode=0100011
Branch=0 MemRead=0 MemtoReg=0 ALUOp=00 MemWrite=1 ALUSrc=1 RegWrite=0

=== BRANCH (BEQ) ===
opcode=1100011
Branch=1 MemRead=0 MemtoReg=0 ALUOp=01 MemWrite=0 ALUSrc=0 RegWrite=0
control_tb.v:82: $finish called at 25000 (1ps)
```

Fig. 7: Control unit output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\Seq Processor\Seq Processor> vvp a.out
time = 11000, A = fffffffffffffffff, B = 0000000000000000, and Cout = 0, S = fffffffffffffffff, carry_flag = 0 overflow_flag = 0, and normal = 0fffffffffffffff
time = 22000, A = fffffffffffffffff, B = 0000000000000001, and Cout = 1, S = 0000000000000000, carry_flag = 1 overflow_flag = 0, and normal = 10000000000000000
time = 33000, A = fffffffffffffffff, B = fffffffffffffffff, and Cout = 1, S = ffffffffffffffffe, carry_flag = 1 overflow_flag = 0, and normal = 1ffffffffffffffffe
time = 44000, A = 8000000000000000, B = 8000000000000000, and Cout = 1, S = 0000000000000000, carry_flag = 1 overflow_flag = 1, and normal = 10000000000000000
time = 55000, A = aaaaaaaaaaaaaaaaa, B = 5555555555555555, and Cout = 0, S = fffffffffffffffff, carry_flag = 0 overflow_flag = 0, and normal = 0fffffffffffffff
time = 66000, A = 000000000000000f, B = 0000000000000001, and Cout = 0, S = 0000000000000010, carry_flag = 0 overflow_flag = 0, and normal = 00000000000000010
time = 77000, A = 0000000000000000f, B = 0000000000000001, and Cout = 0, S = 0000000000000100, carry_flag = 0 overflow_flag = 0, and normal = 00000000000000100
time = 88000, A = 0000000000000fff, B = 0000000000000001, and Cout = 0, S = 0000000000001000, carry_flag = 0 overflow_flag = 0, and normal = 00000000000001000
time = 99000, A = 12153524c0895e81, B = 8484d609b1f05663, and Cout = 0, S = 969a0b2e7279b4e4, carry_flag = 0 overflow_flag = 0, and normal = 0969a0b2e7279b4e4
Passed all given cases
add_oper_tb.v:47: $finish called at 99000 (1ps)
```

Fig. 8: Adder output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog ALU_control.v ALU_control_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
==== ALU CONTROL TEST ====
LOAD/STORE/ADDI -> ADD : ALUControl = 0000 (expected 0000)
BEQ -> SUB : ALUControl = 1000 (expected 1000)
R-TYPE ADD : ALUControl = 0000 (expected 0000)
R-TYPE SUB : ALUControl = 1000 (expected 1000)
R-TYPE AND : ALUControl = 0111 (expected 0111)
R-TYPE OR  : ALUControl = 0110 (expected 0110)
R-TYPE XOR : ALUControl = 0100 (expected 0100)
R-TYPE SLL : ALUControl = 0001 (expected 0001)
R-TYPE SRL : ALUControl = 0101 (expected 0101)
R-TYPE SRA : ALUControl = 1101 (expected 1101)
R-TYPE SLT : ALUControl = 0010 (expected 0010)
R-TYPE SLTU: ALUControl = 0011 (expected 0011)
==== TEST COMPLETE ====
ALU_control_tb.v:106: $finish called at 60000 (1ps)
```

Fig. 9: ALU control output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> iverilog data_mem.v data_mem_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk> vvp a.out
Starting Data Memory Test...
Read data = 1122334455667788
PASS: Read matches written data
Read data = deadbeefcafebabe
PASS
Test complete.
data_mem_tb.v:90: $finish called at 85000 (1ps)
```

Fig. 10: Data Memory output

```
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\Seq Processor\Seq Proces
sor> iverilog mux_64.v mux_tb.v
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\Seq Processor\Seq Proces
sor> vvp a.out
time = 0, I0 = 0, I1 = 0, S0 = 0, Y = 0
time = 10000, I0 = 0, I1 = 0, S0 = 1, Y = 0
time = 20000, I0 = 0, I1 = 1, S0 = 0, Y = 0
time = 30000, I0 = 0, I1 = 1, S0 = 1, Y = 1
time = 40000, I0 = 1, I1 = 0, S0 = 0, Y = 1
time = 50000, I0 = 1, I1 = 0, S0 = 1, Y = 0
time = 60000, I0 = 1, I1 = 1, S0 = 0, Y = 1
time = 70000, I0 = 1, I1 = 1, S0 = 1, Y = 1
mux_tb.v:26: $finish called at 80000 (1ps)
```

Fig. 11: Mux output

```
  A: 4000000000000000  B: 0123400000000001  Opcode: 1101
  Result: 2000000000000000  Zero=0
Test 46:
  A: 0000000000000001  B: 00deed0000000001  Opcode: 1101
  Result: 0000000000000000  Zero=1
Test 47:
  A: 8000000000000000  B: 0000000000000020  Opcode: 1101
  Result: ffffffff80000000  Zero=0
Test 48:
  A: 0000000000000000  B: 0000000000000000  Opcode: 0010
  Result: 0000000000000000  Zero=1
Test 49:
  A: ffffffffffffffff  B: 0000000000000000  Opcode: 0010
  Result: 0000000000000001  Zero=0
Test 50:
  A: 0000000000000000  B: ffffffffffffffff  Opcode: 0010
  Result: 0000000000000000  Zero=1
Test 51:
  A: 8000000000000000  B: ffffffffffffffff  Opcode: 0010
  Result: 0000000000000001  Zero=0
Test 52:
  A: 0000000000000000  B: 0000000000000000  Opcode: 0011
  Result: 0000000000000000  Zero=1
Test 53:
  A: 0000000000000001  B: 0000000000000000  Opcode: 0011
  Result: 0000000000000000  Zero=1
Test 54:
  A: 7fffffffffffffff  B: 8000000000000000  Opcode: 0011
  Result: 0000000000000001  Zero=0
Test 55:
  A: ffffffffffffffff  B: ffffffffffffffff  Opcode: 0011
  Result: 0000000000000000  Zero=1

========================================
  FINAL RESULT: Passed 55/55 tests
========================================

alu_tb.v:324: $finish called at 560000 (1ps)
PS C:\Users\chand\OneDrive\Documents\sem4\IPA\suk>
```

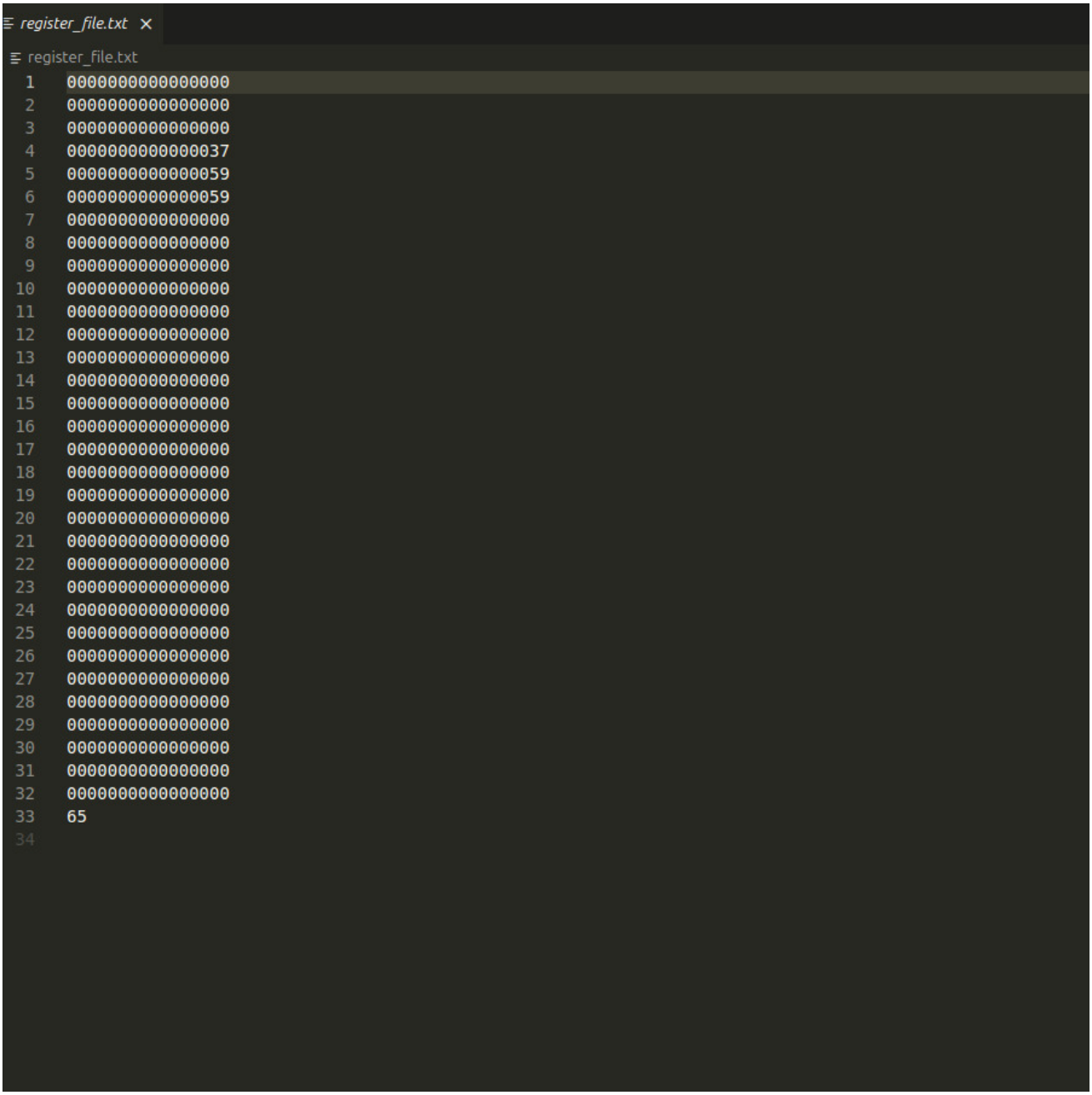
Fig. 12: ALU output

instructions.txt

```
00
a0
00
93
00
00
01
93
00
10
02
13
06
00
84
63
00
41
82
b3
00
40
01
b3
00
50
02
33
ff
f0
80
93
fe
00
06
e3
00
30
30
23
00
00
00
00
```

Fig. 13: Instructions file

To check the working of our final circuit, we implemented a simple Fibonacci algorithm, and the following are the register outputs obtained during execution.



```
register_file.txt
0000000000000000
0000000000000000
0000000000000000
0000000000000037
0000000000000059
0000000000000059
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
65
```

Fig. 14: Register file

V. Challenges Faced

During the design and verification of the sequential RV64I processor, we encountered several practical challenges . Care had to be taken while encoding instructions and extracting fields such as opcode, `funct3`, `funct7`, and immediates for different formats. Generating correctly sign-extended immediates for I-type, S-type, and B-type instructions was particularly error-prone and required careful debugging. Additional issues arose in the control signals for load/store and branch instructions, as well as in ensuring consistent memory addressing and 64-bit alignment. These challenges were addressed iteratively using waveform inspection and targeted test cases. One of the most frequent issues arose while testing branch instructions, that would enter into an infinite loop.

Initially faced the issue of data memory over writing, due to faulty initialisation

Finally, integrating all the modules into the final datapath gave a few errors.

Despite these challenges, working through them significantly improved our understanding of both the RISC-V ISA and the hardware implementation process.

VI. Conclusion and Future Work

This project focused on designing and implementing a sequential 64-bit RV64I RISC-V processor using Verilog. By adopting a single-cycle architecture, we were able to clearly observe how an instruction flows through the five logical stages—fetch, decode, execute, memory access, and write-back—within one clock period. The processor supports a fundamental subset of the RV64I instruction set, including arithmetic, logical, load/store, and branch instructions, and is built as a modular datapath consisting of the Program Counter, Instruction Memory, Register File, Control Unit, Immediate Generator, ALU Control, ALU, Data Memory, and supporting multiplexers and adders.

Each module was implemented and verified independently before integrating them into the complete datapath. Running various RISC-V programs, including arithmetic sequences, memory tests, and a Fibonacci generator, confirmed that the processor executes instructions correctly and produces the expected outputs. These tests validated both the control logic and the datapath behaviour for the instructions we targeted.

Although the current design serves as a clear educational model, there are several directions in which it can be extended and improved. One natural next step is converting the single-cycle processor into a pipelined architecture. Pipelining would allow multiple instructions to be in different stages of execution simultaneously, significantly improving throughput. With pipelining, however, comes the need for hazard handling mechanisms such as forwarding, stalling, and basic branch prediction, which would make the design more realistic and robust.

Another area for improvement is the instruction set coverage. The present processor supports only a small subset of RV64I instructions. Extending support to additional arithmetic operations, logical instructions, shift variants, and jump instructions would make the processor more complete. Enhancing the memory system by adding instruction and data caches would also reduce memory access latency and bring the design closer to real-world processors.

Additional improvements may include optimizing critical-path delay, integrating exception and interrupt handling, and exploring multi-cycle implementations to ease timing constraints. Each of these extensions would build on the solid foundation of the sequential design and move the processor from a basic academic model toward a more capable and performance-oriented architecture.

Overall, this project provided hands-on experience with digital design, Verilog implementation, and the RISC-V ISA. The sequential processor forms a strong base for further experimentation, deeper architectural exploration, and future enhancements.